

THE EVOLUTION OF AI MEMORY · RNNs TO TRANSFORMERS

AI didn't get smarter. It just stopped forgetting.

Every AI architecture solved one problem, and created the next. This is that story.

Dibayendu Mukherjee

AI Research Engineer · Systems Engineer

Why Does AI Need Memory?

Read this sentence:

"The cat chased the dog because it was hungry."

Who is "it"?

Most people answer "the cat" without thinking.

Because your brain remembers what it read just moments ago.

Early neural networks had no reliable way to connect "it" back to "the cat." To early neural networks, every word looked almost independent from the last.

Without a way to remember previous words, pronouns like "it" became surprisingly difficult.

The Real-World Gap

Whenever order carries meaning, language, speech, music, DNA, or stock prices, the model needs to remember what came before. The obvious question became: How do we teach a neural network to remember? Because Without memory, it can't understand context. The industry desperately needed a model that could remember the past.

The First Attempt: Teaching AI to Remember

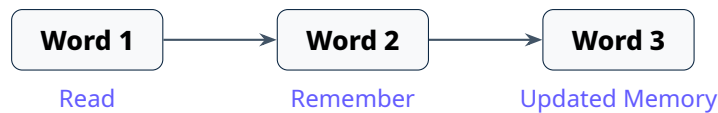
If forgetting is the problem, how would you solve it?

Think about reading a book.

You don't memorize every word you've read. Instead, your brain keeps a running summary of the story so far.

Engineers borrowed this exact idea when they designed the **RNN (Recurrent Neural Network)**.

The word "recurrent" simply means "repeating." The same neural network is reused over and over, once for every word in the sequence.



Instead of reading a sentence all at once, the RNN reads word-by-word. It takes the new word, mixes it with its "running summary" from the previous step, and updates the summary. That updated memory is then passed to the next word.

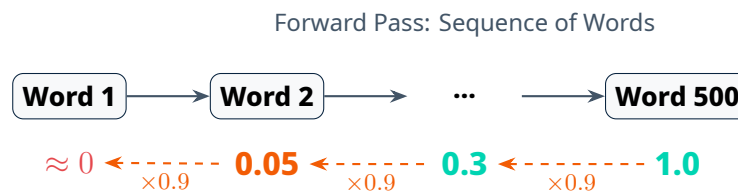
It was a brilliant fix. For short sentences, it worked remarkably well.

But for long ones... everything started falling apart.

The Flaw: Vanishing Gradients

What if the sentence is 500 words long? The real failure happens under the hood *during training*.

To learn, the network sends a learning signal (gradient) backward through time. Each step multiplies this signal by a weight matrix. For a 500-word sequence, that's 500 multiplications. Unless perfectly balanced, the signal either shrinks to zero (vanishing) or grows out of control (exploding). This exponential instability makes it mathematically impossible to adjust weights for early words.



Backward Pass: The Shrinking Learning Signal

If the multiplier is even slightly less than 1, the math is brutal: $0.9^{100} \approx 0.000026$. The gradient becomes practically zero. Imagine whispering a sentence across 300 people, each speaking slightly quieter than the last. Eventually, the final person hears nothing. The RNN's learning signal weakens exactly like this, fading away before it can teach the network that the first word mattered.

The Realization

The problem wasn't that the network had memory. The problem was that it couldn't learn how to preserve memories over long sequences because the training signal vanished before reaching earlier words.

To understand long documents, AI needed a mechanism allowing important information to survive hundreds of timesteps.

That idea became the Long Short-Term Memory (LSTM) network.

The Second Idea: Controlled Memory

The problem was not that RNNs had no memory.

The problem was that important memories could not survive because the learning signal disappeared across long sequences.

The solution:

Give the network a way to **control memory flow**.

Enter the **LSTM (Long Short-Term Memory)**.

The Core Idea:

Instead of one fragile memory, LSTM creates a protected memory highway called the **cell state**.

Think of it as a conveyor belt carrying important information through time.

But unlike a normal conveyor belt, the network has learned gates that decide what passes through.

Forget Gate

Removes information that is no longer useful.
"Should this old memory still matter?"

Input Gate

Adds new important information.
"Should this new information be stored?"

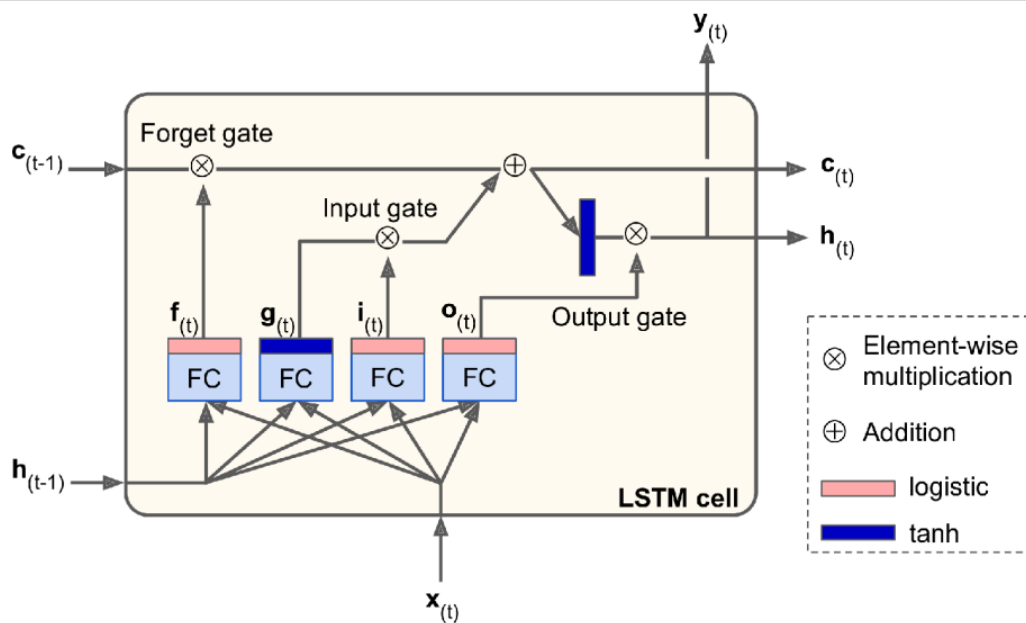
Output Gate

Controls what memory is revealed.
"What information should influence the current decision?"

LSTM did not make neural networks remember everything.
It made them learn what deserves to be remembered.

Looking Under the Hood

Let's map that factory analogy to the actual architecture.



The top horizontal line is the **Cell State**, the protected memory highway. Unlike a normal RNN, this pathway allows information to travel across many time steps with less degradation.

The breakthrough was not just adding gates. LSTM created a protected path for information and gradients to flow through long sequences, becoming the dominant sequence architecture before the rise of attention and transformers.

Inside the Gates

The gates are small neural networks that use logistic (sigmoid) functions to create values between 0 and 1:

Forget Gate

Decides how much old information should remain.

Alice → Bob context reduced.

Input Gate

Decides what new information deserves to be written into long-term memory.

Output Gate

Decides which part of memory should influence the current prediction.

The Mathematical Formulation:

$$\mathbf{f}_{(t)} = \sigma(\mathbf{W}_{xf}^T \mathbf{x}_{(t)} + \mathbf{W}_{hf}^T \mathbf{h}_{(t-1)} + \mathbf{b}_f)$$

$$\mathbf{i}_{(t)} = \sigma(\mathbf{W}_{xi}^T \mathbf{x}_{(t)} + \mathbf{W}_{hi}^T \mathbf{h}_{(t-1)} + \mathbf{b}_i)$$

$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \tanh(\mathbf{W}_{xg}^T \mathbf{x}_{(t)} + \mathbf{W}_{hg}^T \mathbf{h}_{(t-1)} + \mathbf{b}_g)$$

Note: $\mathbf{W}$ represents weight matrices, $\mathbf{b}$ bias terms, σ the sigmoid gate controller, and $\otimes$ element-wise multiplication.

The New Wall: Sequential Processing

Did LSTMs solve everything?

They solved a major problem: keeping important information alive.

But a new limitation appeared.

The model still had to read one word at a time.

The Sequential Bottleneck

An LSTM cannot process word #100 before finishing word #99.

$$Word_1 \rightarrow Word_2 \rightarrow Word_3 \rightarrow \dots \rightarrow Word_{100}$$

Each step depends on the previous hidden state.

GPUs are designed for massive parallel computation, but the sequence dependency prevents processing all words simultaneously.

Imagine a factory where every worker needs the previous worker's result before starting.

The factory is powerful, but the workflow is forced into a single line.

The next question became:

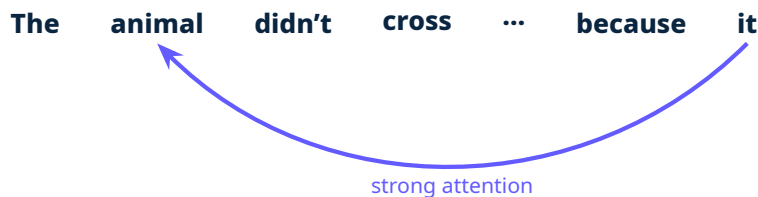
Can AI understand relationships between all words at once?

This question led to the invention of **Attention**.

The Spark: Skipping the Line

Humans do not process every word equally. Instead, we naturally focus on the words that matter for understanding the current one. *"The animal didn't cross the road because it was tired."* When we see "it", our brain immediately looks back at "animal". AI borrows this intuition through a mechanism called **Attention**.

The New Idea: Instead of compressing the entire sentence into a single memory vector, keep every word available. Whenever new information is needed, look back directly at the most relevant words.



Instead of forcing information to travel step-by-step through every intermediate word, attention lets the model directly connect the current word with the most relevant words, regardless of their distance.

The key question is: **How can a model decide which previous words are important?**

Attention Scores

For every word being predicted, the model assigns a score to every word in the input sentence. Higher scores mean "pay more attention." Lower scores mean "this word is less useful."

The next question is: **How are these attention scores actually computed?**

The Mechanics: Query, Key, Value

To compute an attention score, the AI uses a concept borrowed from database retrieval: **Query, Key, and Value**.

Imagine you go to a library looking for a book about "Neural Networks."

The Library Analogy

- **Query (Q):** What you are looking for.
("I need a book on Neural Networks.")
- **Key (K):** What the library has.
(The title on the spine of every book.)
- **Value (V):** The actual content.
(The pages inside the book you ultimately read.)

In a sentence, every word acts as all three simultaneously!

When predicting the word "it":

- "it" sends out a **Query**: *"I am a pronoun looking for a noun."*
- Each previous word holds up a **Key**: *"I am an animal.", "I am a road."*
- The model measures how well the Query matches each Key.
- Based on the strongest matches, the model retrieves the corresponding **Value**.
- The Value defines how much information from that word should be used.

The Math: Scaled Dot-Product

How does the math actually compare a Query and a Key?

Through a **Dot Product**. In linear algebra, a dot product measures how similar two vectors are. If the Query ("pronoun looking for noun") and the Key ("animal") point in the same direction, their dot product is a large number.

The Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Step-by-step translation:

1. QK^T : Compare the Query with every Key through **matrix multiplication** to produce raw similarity scores.
2. **Divide by** $\sqrt{d_k}$: Scale the scores so they remain numerically stable during training.
3. **softmax**: Convert the raw scores into attention weights (probabilities) that sum to 1. (e.g., 90% attention to "animal", 10% to "road").
4. **Multiply by** V : Use those attention weights to compute a weighted combination of the Value vectors.
5. Q , K , and V : These are learned projections of the original word embeddings (vector representation of the words). Queries ask, "What am I looking for?", Keys answer, "What information do I contain?", and Values hold that information. Queries and Keys live in the same vector space so they can be meaningfully compared, while Values are combined according to the resulting attention weights.

This math happens instantly, drawing perfect connections across the sentence.

The Paradigm Shift: Transformers

Now that we understand how Attention directly connects words, we reach a historical turning point. Initially, attention was introduced as an additional component on top of recurrent networks like LSTMs. Once researchers realized that attention—not recurrence—was responsible for most of the model's ability to capture long-range relationships, they began asking whether recurrence was needed at all. In 2017, the paper **Attention Is All You Need** proposed a bold question: *Can we remove recurrence completely?*

Remove sequential computation. Process all tokens in parallel.

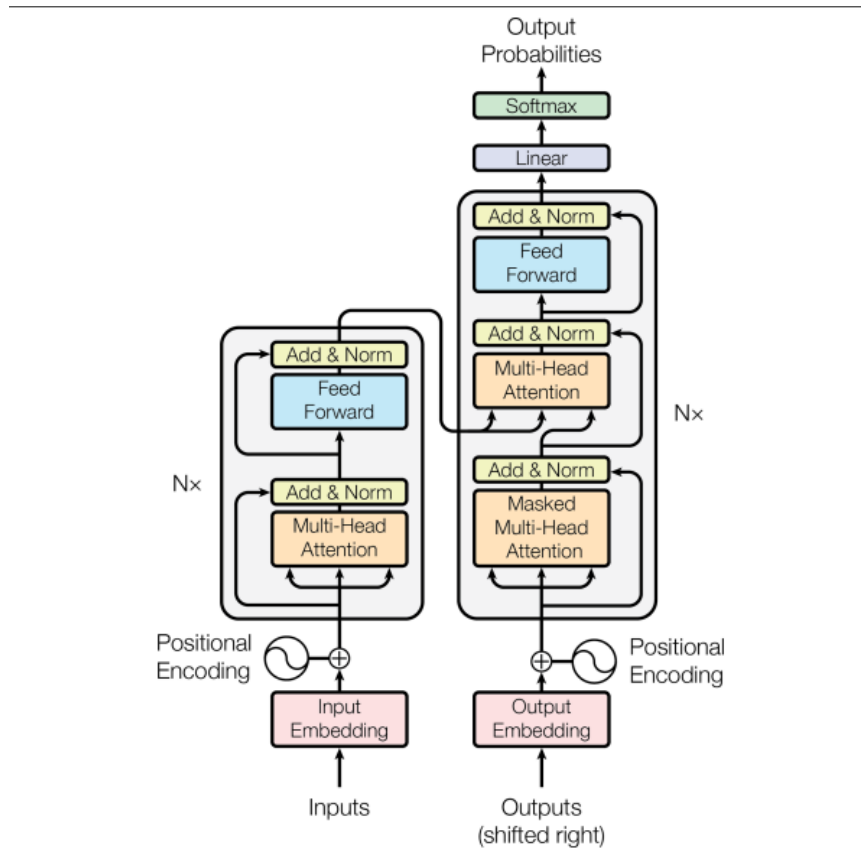
The Cocktail Party Analogy: In an LSTM, 100 words stand in a line playing telephone. In a Transformer, all 100 words are in the same room. Instead of maintaining a single hidden memory that moves through the sentence, every word now has direct access to every other word. At the exact same second, every word computes how relevant every other word is **for the current layer**. The word "it" immediately looks back at "animal," instantly connecting across the room.

The goal is still the same: understand relationships between words. Only the method has changed. Since every word is processed independently during this step, modern GPUs can compute many attention operations simultaneously. This parallel computation makes Transformers dramatically faster to train on modern GPUs.

Removing recurrence solved one problem, but it also introduced a new question: **If there is no loop anymore, what exactly is happening inside a Transformer block?**

How Transformers Actually Work (Part 1)

A Transformer replaces the step-by-step processing of RNNs with a stack of parallel processing blocks.



- **Input Embedding:** Words are converted into numbers (vectors) that the neural network can understand. Similar meanings tend to have similar positions in this mathematical space.
- **Positional Encoding:** Since all words are processed together, the Transformer needs extra information about word order. Positional encoding tells the model:

"This word is first, this word is second, this word is third..."

Without it, the model would treat: *"Dog bites man"* and *"Man bites dog"* as the same collection of words.

- **Multi-Head Attention:** The AI calculates attention scores for every word simultaneously. "Multi-Head" means it looks for different types of connections. Different attention heads often learn to capture different kinds of relationships, such as syntax, long-range dependencies, or coreference, although these roles are learned automatically rather than assigned beforehand.

How Transformers Actually Work

After attention connects words together, the Transformer refines this information through repeated layers.

- **Feed Forward Network:** After attention gathers information from other words, each word representation is independently transformed and refined using a small neural network.
- **Add & Norm:** These connections help information flow through many layers.
 - **Add (Residual Connection):** Keeps the original information and adds the new learned information.
 - **Normalization:** Keeps values stable so training becomes easier.
- **Encoder Stack:** The encoder repeatedly applies attention and transformation to build a deeper understanding of the input sentence.
- **Decoder Stack:** The decoder uses this understanding to generate output one part at a time, while using masked attention to avoid seeing future words.

The Big Idea:

Instead of forcing information to travel step-by-step through a chain, Transformers create direct connections between all words and learn which connections matter.

The Catch: Why We Aren't Done

Transformers have a massive memory problem. In self-attention, every word compares itself with every other word to decide which connections matter.

$$\text{Number of comparisons} = n \times n = n^2$$

Here, n represents the sequence length (number of words/tokens).

Self-attention complexity: $O(n^2)$

This means the cost grows quadratically:

Sequence Length	Attention Comparisons
1,000 tokens	1,000,000 comparisons
10,000 tokens	100,000,000 comparisons

Doubling the sequence length roughly quadruples the computation and memory.

This memory cost grows extremely fast, limiting how much text early Transformers could process at once.

What comes next?

To solve this bottleneck, researchers are exploring architectures like State Space Models (SSMs) and Mamba, which aim for closer to linear-time scaling while keeping the ability to model long-range relationships.

The cycle continues.

THE BIG TAKEAWAY

Hardware always wins.

Transformers didn't win because they were simply smarter. They won because they matched the strengths of modern hardware.

Building AI systems by understanding the architectures behind them.

GitHub: github.com/MrHeavenly
Portfolio: <https://mrheavenly.github.io/>
LinkedIn: [linkedin.com/in/dibayendu-mukherjee-bb897b267](https://www.linkedin.com/in/dibayendu-mukherjee-bb897b267)

Dibayendu Mukherjee